Satellite imagery segmentation system for risk estimation

Santuzz (Davide Santoli), SiceX (Sice), Nicolò Zanoni

Abstract

In this project we present a possible approach to satellite imagery analysis. In recent years satellite imagery has become a significant source of data that can be used to achieve a wide range of tasks, e.g. risk assessment, change detection, mapping, damage analysis. Deep learning has led to an improvement of the analysis and interpretation of satellite imagery, allowing to automatize and make more efficient all the processes connected to this field. The purpose of this project was to build a deep learning system able to recognize the different classes of land present in satellite images and perform a risk estimation analysis based on it.

1. Introduction

The main objective of the project was to perform a risk analysis based on the information gathered from satellite imagery. To do this, the first step is to identify the classes of environments present in each image. Some of these environments were considered risk areas, such as lakes, rivers and forests, as they can be the source of floods or fires. This kind of analysis can be especially useful if done automatically as it permits to get an updated mapping of the risk areas, to understand how serious the effect of natural calamities on populated areas can be, and to get all the necessary measures for the safety of the population.

2. Datasets

In this project we used 2 different datasets: firstly, we started working with the DeepGlobeLandCover [1] dataset; then we integrated it with the LoveDA [2] dataset in order to provide more training material for our segmentation component.

DeepGlobeLandCover:

The DeepGlobeLandCover Dataset contains 803 satellite images in RGB format, of size 2448x2448. The photos have a 50cm pixel resolution, collected by DigitalGlobe's satellite. The dataset is composed of images and masks and split in a training set of 683 images and a test set of 120. The training set was further augmented by half (341 images), as we will discuss later, bringing this train set to 1024 images in total.
The masks are RGB images with 7 classes of labels, using color-coding (R, G, B) as follows:

- Urban land: (0, 255, 255) – cyan: Man-made, built up areas with human artifacts (ignoring roads for now as they are hard to detect);
- Agricultural land: (255, 255, 0) - yellow: Farms, any planned (i.e. regular) plantation, cropland, orchards, vineyards, ornamental horticultural areas, confined feeding operations;
- Rangeland: (255, 0, 255) – pink: Any non-forest, non-farm, green land, grass;
- Forest land: (0, 255, 0) – green: Any land with x% tree crown density plus clearcuts;
- Water: (0, 0, 255) - Rivers, oceans, lakes, wetland;
- Barren land: (255, 255, 255) - Mountain, land, rock, desert, beach, no vegetation;
- Unknown: (0, 0, 0) - Clouds and others.

This dataset was found to be quite unbalanced in its share of the seven classes:

- urban_land: 81.64%

- agriculture_land: 89.74%
- rangeland: 63.57%
- forest_land: 24.31%
- water: 59.17%
- barren_land: 52.44%
- unknown: 37.10%

So we tried to alleviate this by applying data augmentation and merging it with the LoveDA dataset.

LoveDA:

This dataset is composed of 5987 1024x1024 satellite photos of 18 districts of the cities Nanjing, Changzhou, and Wuhan, with a spatial resolution of 0.3 meters. Of these, 2522 make up the train set, the only ones with annotation masks. As during the training we also needed annotations in the test set, we chose to only integrate the LoveDA train set in our final dataset.
The LoveDA dataset contains the following classes:

- Background
- Forest
- Agriculture
- Barren
- Road
- Building
- Unknown

In the integration with the other dataset, the 'road' class in the masks was remapped to 'unknown' as the DeepGlobeLandCover one lacks such a class and that could have led to further balancing problems during the training of our network.

Final dataset:

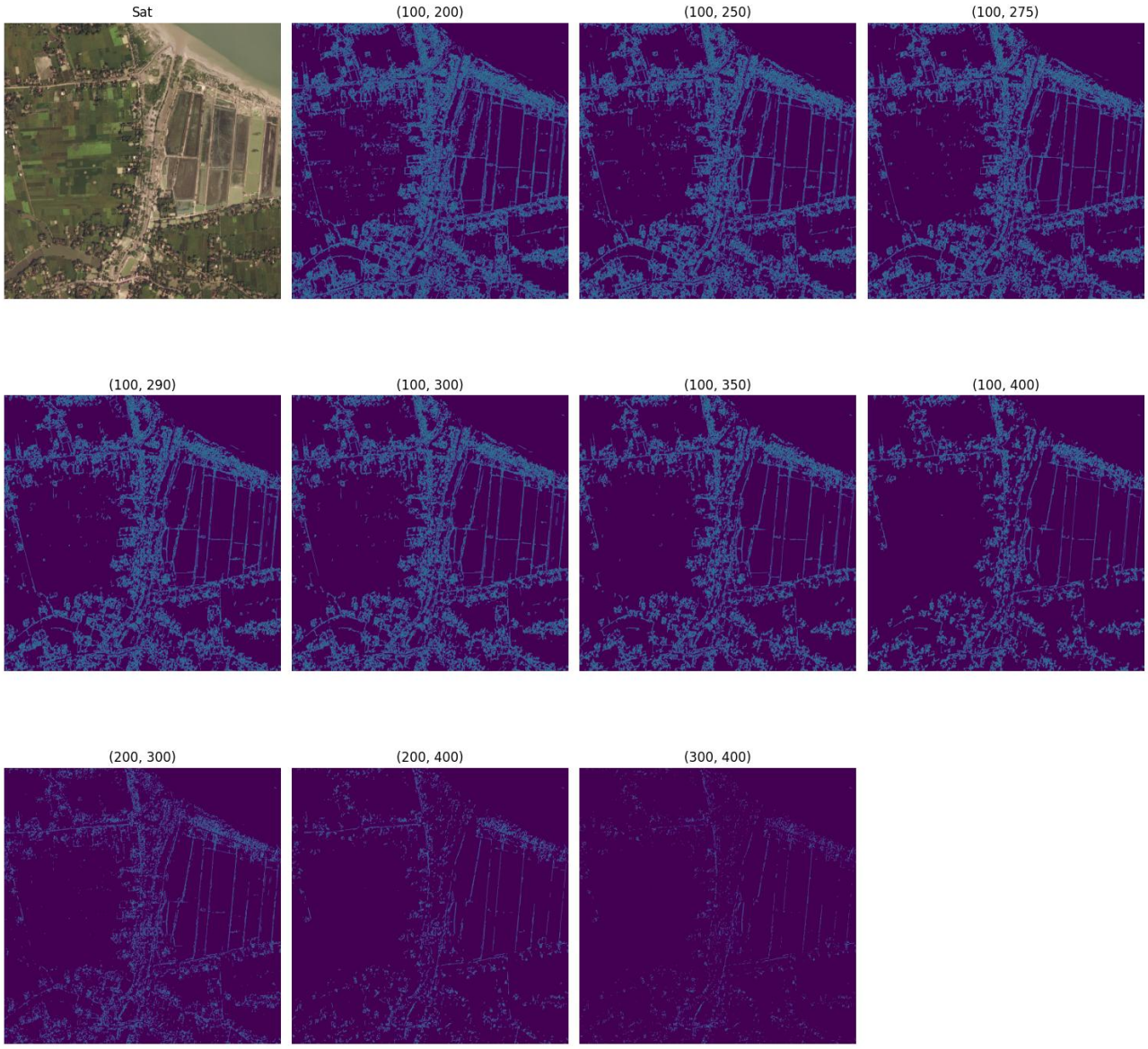
The final dataset used to train the segmentation component is composed of 3662 images of two sizes, 2448x2448 and 1024x1024. The train set counts 3237 (683 from base DGLC, 341 augmented from DGLC, 2213 from LoveDA) images while the test set 425 (120 DGLC testset, 305 moved over from the LoveDA train set), and it now has the following class distribution:

- urban_land: 90.96%
- agriculture_land: 94.63%
- rangeland: 80.08%
- forest_land: 60.76%
- water: 77.10%
- barren_land: 60.72%
- unknown: 73.80%

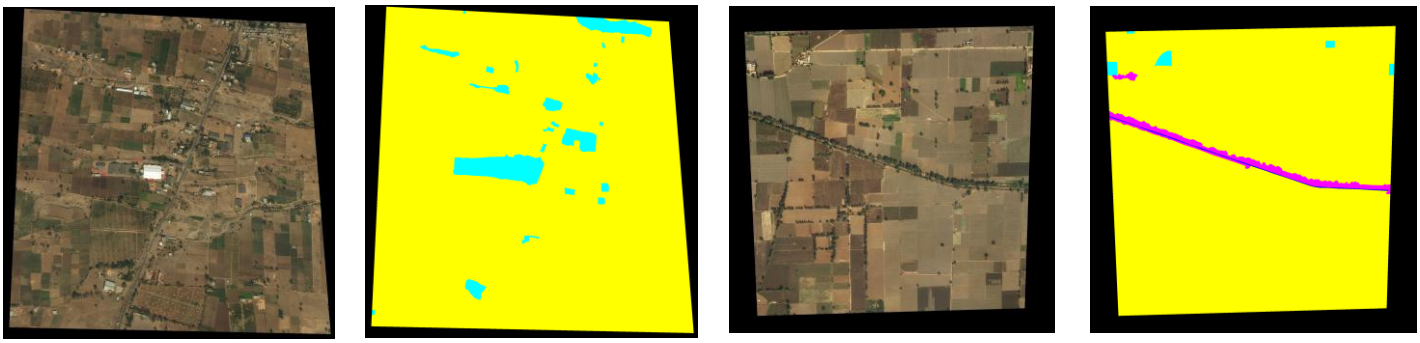
3. Preprocessing and data augmentation

The first step of the pipeline was to apply data augmentation and geometric transformations.

Our first goal was to detect the edges of both training and test datasets in order to have a grayscale edge mask of every image. These masks would then be added as the fourth alpha channel to the original RGB images, making them RGBA png images. We thought feeding the network with an additional channel of data representing the objects' edges could help it to better distinguish between the shapes of the buildings from those of forest canopies or farmlands.
The edges were calculated using the Canny [6] algorithm with thresholds of 100 and 200. Other thresholds were tried but these ones proved to be better at highlighting the features we wanted.



Having generated our additional data channel, we then performed data augmentation by applying random perspective transformations using the RandomPerspective function from the torchvision library. After some testing we settled on a light 0.2 distortion scale to simulate a photo taken from a satellite slightly off-position. We used nearest neighbor interpolation to avoid artifacts in the segmentation masks.
The augmentation was applied only on the DGLC train set to better balance it with the considerably larger LoveDA dataset.



4. Segmentation

To extract the semantic segmentation from the satellite imagery we implemented a modified version of the Segformer Neural Network [3]. The original Segformer is an encoder-decoder architecture that takes as input RGB or RGBA images. The modified version of Segformer is composed by two components:

- A backbone called Group Vision Transformer, which is a modified version of the original encoder part;
- A decode head called Edges-RGB Segformer Head.

Encoder: Group Vision Transformer

The first step performed by this component is to split RGB channels from the edge channel. In both cases, every image is firstly divided in patches which are overlapped so that information is shared between them. Then, the encoder is divided in two parts: one for RGB channels, the other for the edge channel. In the RGB part, patches pass through a series of 4 transformer blocks; in the edges part, which is the main modification with respect to the original model, we tried different configurations with 1, 2 and 4 transformer blocks. A transformer block is composed of two fundamental layers: an Efficient Self attention and a Mix-FeedForward Network.

Efficient self-attention: it differs from the original self-attentive mechanism by reducing the dimensionality of K and V by a ratio R, reducing the time complexity from (N^2) to O(N^2/R).

Mix-FeedForward Network: since the model does not use positional encoding to avoid accuracy problems related to semantic segmentation with different resolutions, it has a Mix-FFN which uses a 3x3 convolution for the feed-forward network in order to learn the spatial relationships between pixels which is leaked through zero padding.

Decoder: Edge-RGB Segformer Head:

The original Segformer head is composed of four parts:

1. Multi-level features from the GiT go through an MLP layer to unify the channel dimension
2. Features are up-sampled and concatenated together, with the final output dimension becoming (H/4)x(W/4)x(5*C)
3. Another MLP layer is used to fuse the concatenated features
4. The final MLP layer takes the fused feature to predict the segmentation mask, which has a resolution of (H/4)x(W/4)xNum_classes, with Num_classes = 8 in our case.

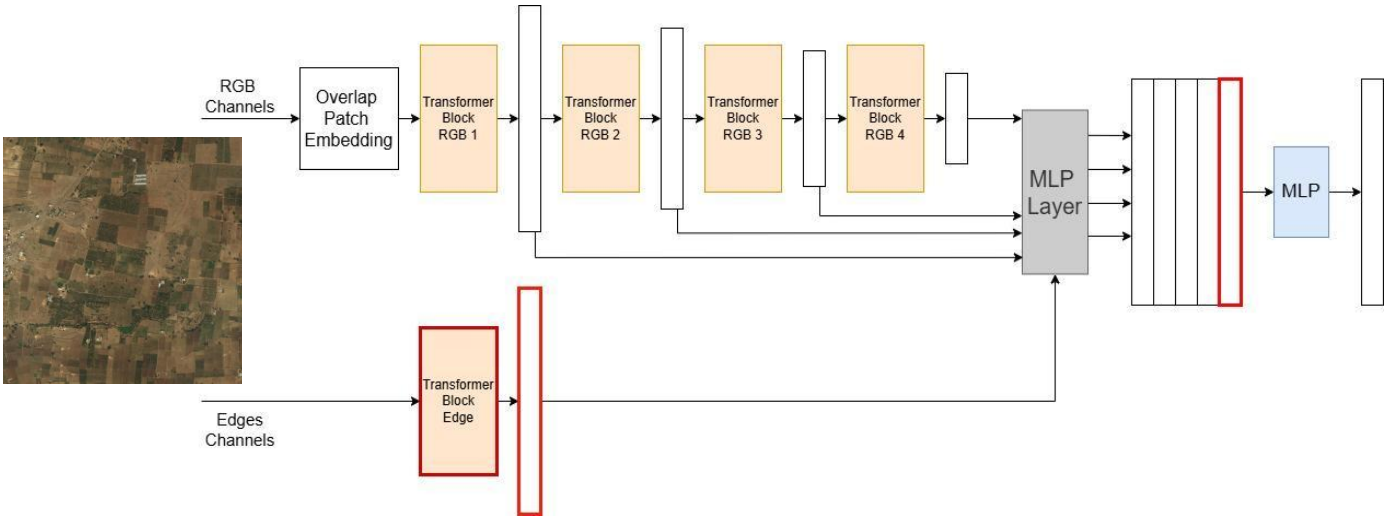


Figure 1: Modified Segformer architecture

The model which performed best in our experiments was the model with 1 transformer block for the edge channels, shown in Figure 1 with modifications applied to the original Segformer model underlined in red. These changes were made to let the Segformer be able to take in input an additional channel, which in our case was the edge channel.

5. Risk assessment

The final goal of the project was to estimate the risk of flooding and forest fires in a given populated area by analyzing the segmented images given by the modified Segformer.

In the case of flooding, risk estimation is based on two measures taken from [4]:

- Elevation of populated areas
- Distance between populated areas and the risk areas

In the case of forest fires the estimate is based on elevation categories taken from [5] and the same distance categories of [4].

These are the distance distance categories for both risk types:

- 0: >8614m;
- 1: 3000-8614(m);
- 2: 2000-3000(m);
- 3: 1000-2000(m);
- 4: 500-1000(m);
- 5: 0-500(m)

These are the elevation categories for flooding:

- 0: >2821(m);
- 1: 488-2821(m);
- 2: 339-488(m);
- 3: 219-339(m);
- 4: 118-219(m);
- 5: 0-118(m)

And these are the elevation categories for forest fires:

- 0: >900m;
- 1: 800–900(m);
- 2: 700–800(m);
- 3: 600–700(m);
- 4: 500–600(m);
- 5: 0–500(m)

Populated areas are represented by the urban area class, while risk areas can be rivers, lakes and forests. The final risk level in a given pixel of urban area is calculated as:

$$R = \alpha * distance \ + \ (1 - \alpha) * elevation$$

Where α is the weight of each measure (we chose to use 0.5) and $distance$ is measured to the nearest pixel of the relevant risk area. The urban area in the image is then color-coded based on their calculated vulnerability as follows:

- Safe:
- Very low:
- Low:
- Moderate:
- High:
- Very high:



6. Image retrieval

The image retrieval component was implemented following an unsupervised approach to retrieve, given an input satellite image, a list of the most similar images present in the training dataset. The system was built to search those images that contained instances of the classes present and a similar environment with respect to the input image.

The retrieval component of our project is composed of two parts: in the first part we built a convolutional autoencoder and we trained it on the whole training dataset to extract feature maps of all its images. Our Convolutional Autoencoder is composed of a combination of Convolutional Layers, SELU and Sigmoid activation functions.

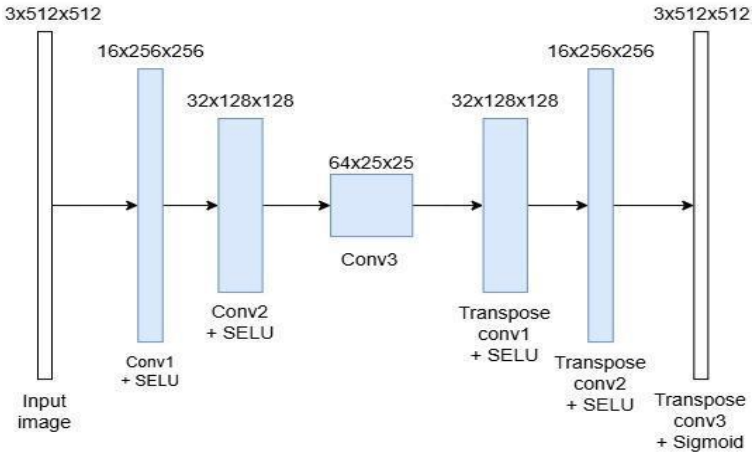


Figure 2: Autoencoder architecture

In the second part we use a combination of Nearest Neighbor algorithm and histogram similarities computation to retrieve, given a test image in input, the n most similar images in the training dataset.

After training, the decoder part was detached and the encoder was used to extract feature maps of the whole training dataset. In this way we built an embedding of dimensions num_imagesx64x25x25.

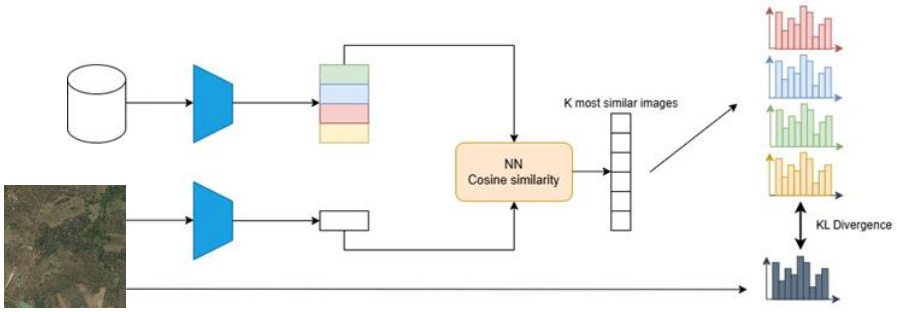


Figure 3: Retrieval architecture for inference

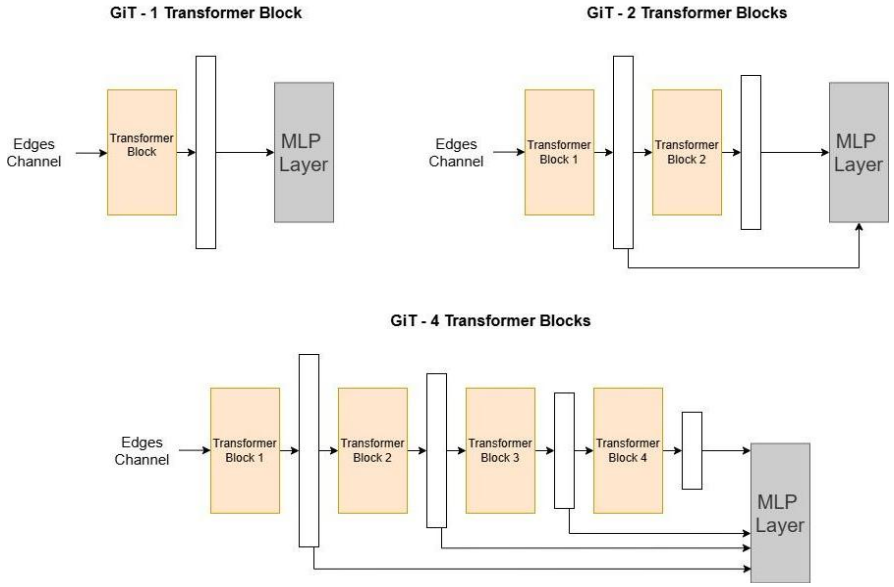
During test, we used the same encoder to extract the feature map of a test image, and we applied the Nearest Neighbour algorithm in order to find the top k similar images in the training dataset; the similarity was given by computing cosine similarity between the feature maps of both the input image and every image from the training dataset. Then, to obtain a better result, for every image given in output by the Nearest Neighbour we computed the 3 color histograms (one for every RGB channel) and we computed the KL divergence between them and the color histograms of the input image.

Finally, images were ordered using the value of the computed KL divergence, and we took the first n<k images with the most similar histograms to the input's ones as the output of the retrieval component.

7. Experimental evaluation

7.1 Segmentation

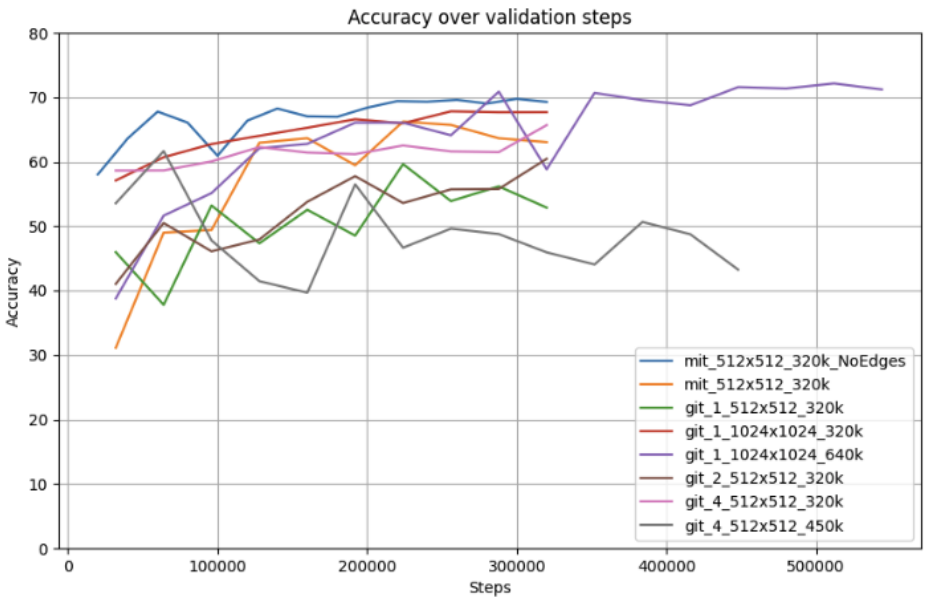
A number of experiments have been carried out in order to find the best possible segmentation models to achieve the main objective of our project. We firstly used the original Segformer version in order to have a reference result for comparison with our modified version of the model. We evaluated 3 different models which differ in the number of transformer blocks used for the edge channel.

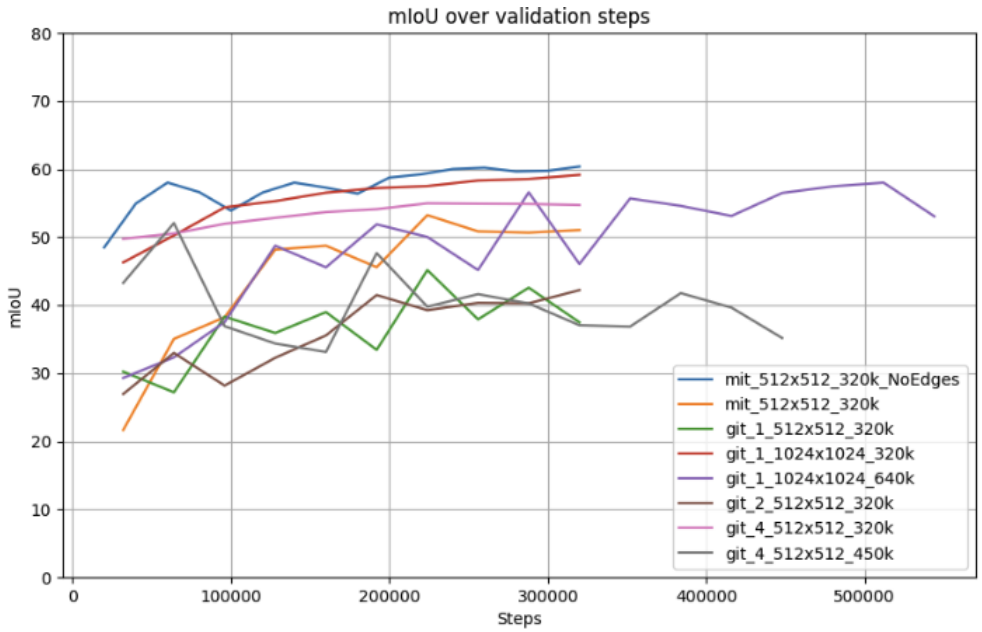


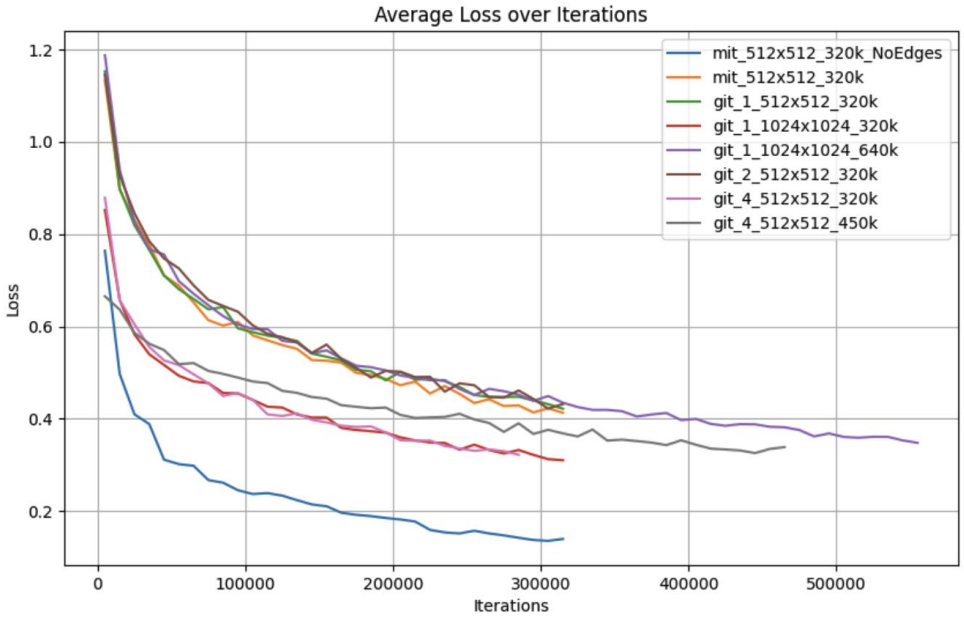
The 3 models were trained with varying input image resolutions and number of iterations, trying to find the best configuration for our purpose. The MiT (Mixin Vision Transformer) model is the original used by the Segformer architecture.

Model	Image resolution	Number of iterations	Edges	# Transformer blocks (edge channel)
MiT	512x512	320’000	Yes	-
MiT	512x512	320’000	No	-
GiT_1	512x512	320’000	Yes	1
GiT_1	1024x1024	320’000	Yes	1
GiT_1	1024x1024	640’000	Yes	1
GiT_2	512x512	320’000	Yes	2
GiT_4	512x512	320’000	Yes	4
GiT_4	512x512	450’000	Yes	4

Models were evaluated by computing 2 different metrics: Accuracy and IoU (Intersection over Union).
The following charts represent the trend of the two metrics with respect to the number of training steps.







The following table shows for each model the performance metrics on each class.

	GiT_1 320k 512x512		GiT_1 320k 1024x1024		**Git_1 640k 1024x1024**		GiT_2 320k 512x512		GiT_4 320k 512x512		GiT_4 450k 512x512	
Class/metric	IoU	Acc	IoU	Acc	IoU	Acc	IoU	Acc	IoU	Acc	IoU	Acc
Urban land	54.19	87.13	65.95	85.14	**72.02**	**84.81**	55.63	77.16	2.8	2.81	6.34	6.41
Agriculture land	59.14	76.25	79.69	89.31	**84.26**	**93.82**	73.31	80.16	70.76	94.92	52.76	99.21
Rangeland	18.49	47.32	26.97	38.03	**34.73**	**42.93**	19.81	53.54	33.32	45.18	2.2	2.27
Forest land	24.39	26.23	71.12	86.65	**78.58**	**92.44**	68.91	87.61	76.94	85.76	13.24	13.69
Water	41.64	58.05	67.69	71.4	**75.51**	**81.95**	40.16	43.33	65.61	71.23	6.17	6.26

Barren land	26.35	30.75	59.99	73.61	**68.9**	**77.91**	25.89	28.43	59.36	78.52	4.01	4.08

The overall best results among the models we trained were achieved by the GiT_1 version, trained with 1024x1024 images in 640’000 iterations. At inference time, the model was used on some satellite images that we took from Google Earth Engine in order to also evaluate the model on images taken from outside the datasets.

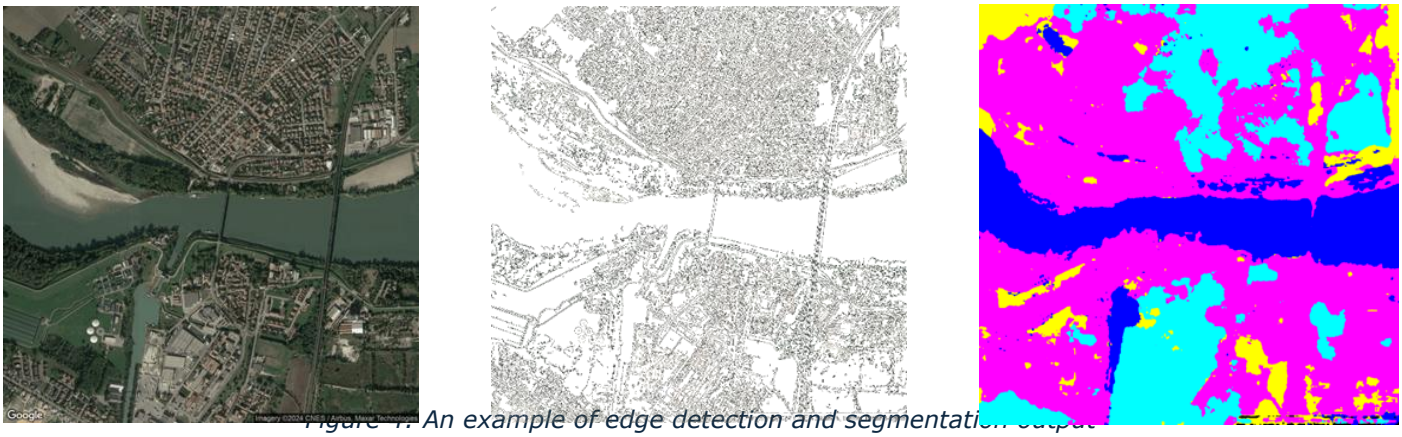


Figure 4: An example of edge detection and segmentation output

7.1.2 Risk assessment

We then developed a small script which takes in input any geographic coordinates to query the Maps Static API and retrieve the satellite image, as well as using the Elevation API to generate a coarse 16x16 elevation map of the area of interest.
The image is then processed by the model and the segmented output is used to calculate a flood risk map and a fire risk map.

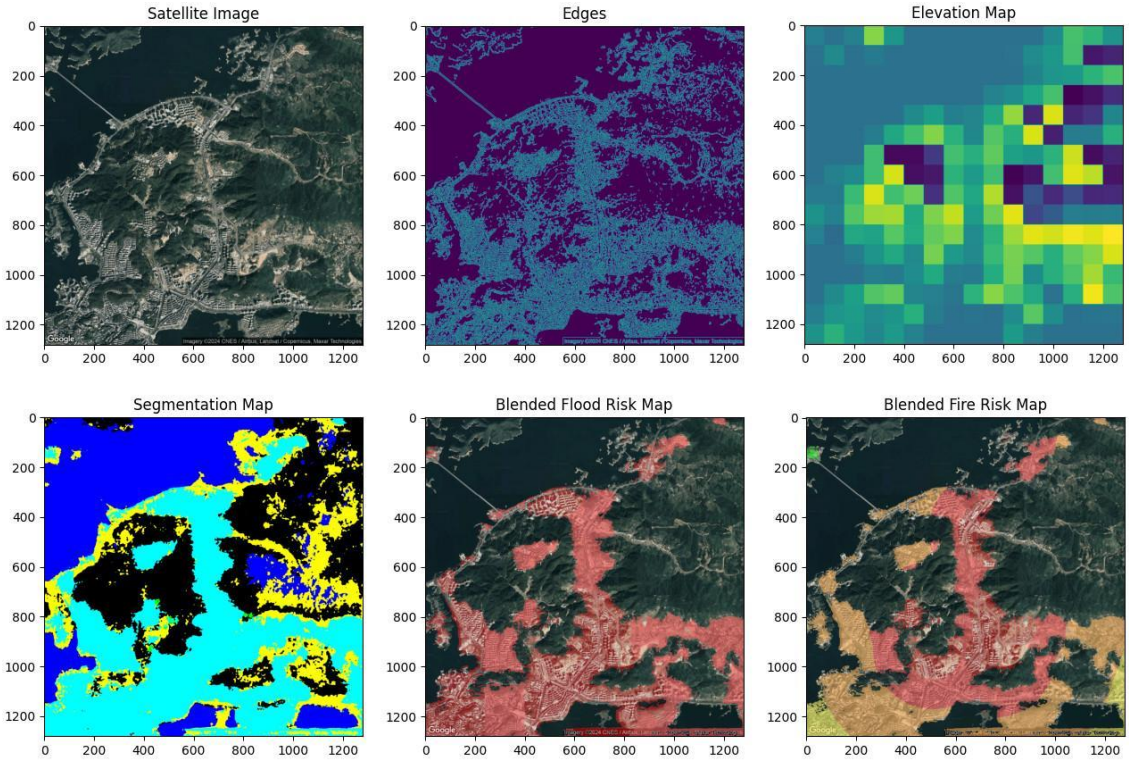


Figure 5: Chun’An, China. The model performed badly at classifying forests, but better with urban and water.

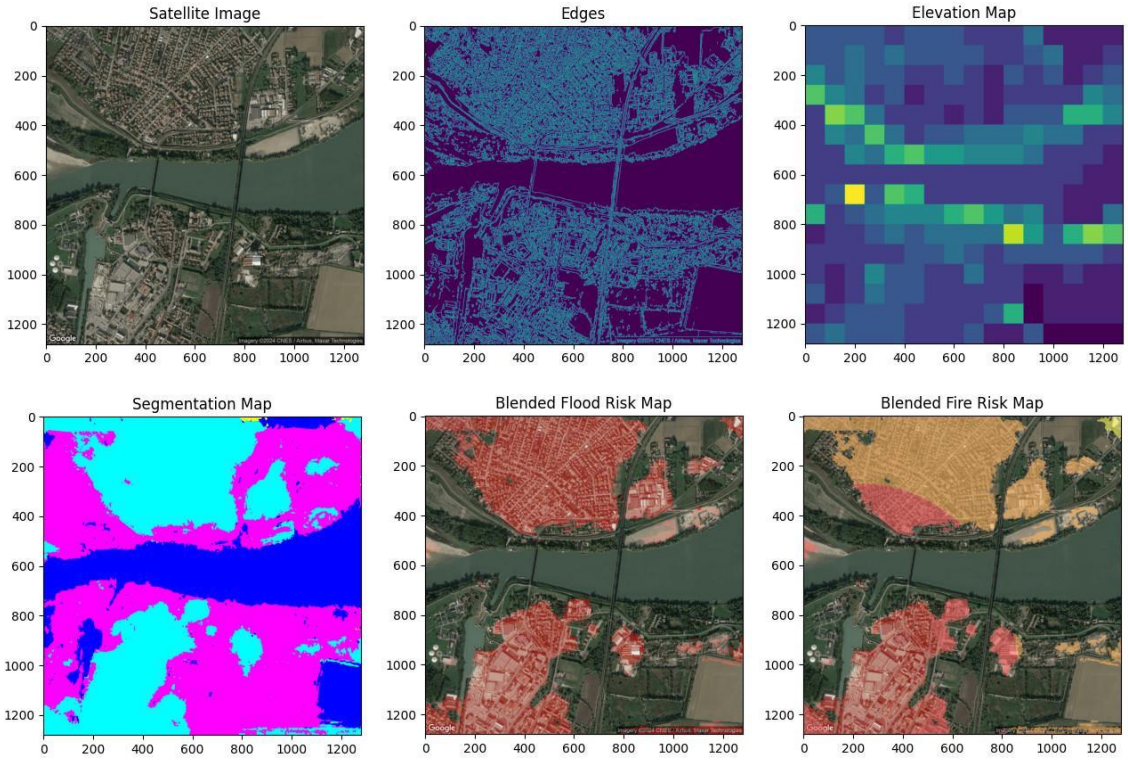


Figure 6: Pontelagoscuro, Italy. The river is correctly classified but there are some false positives.

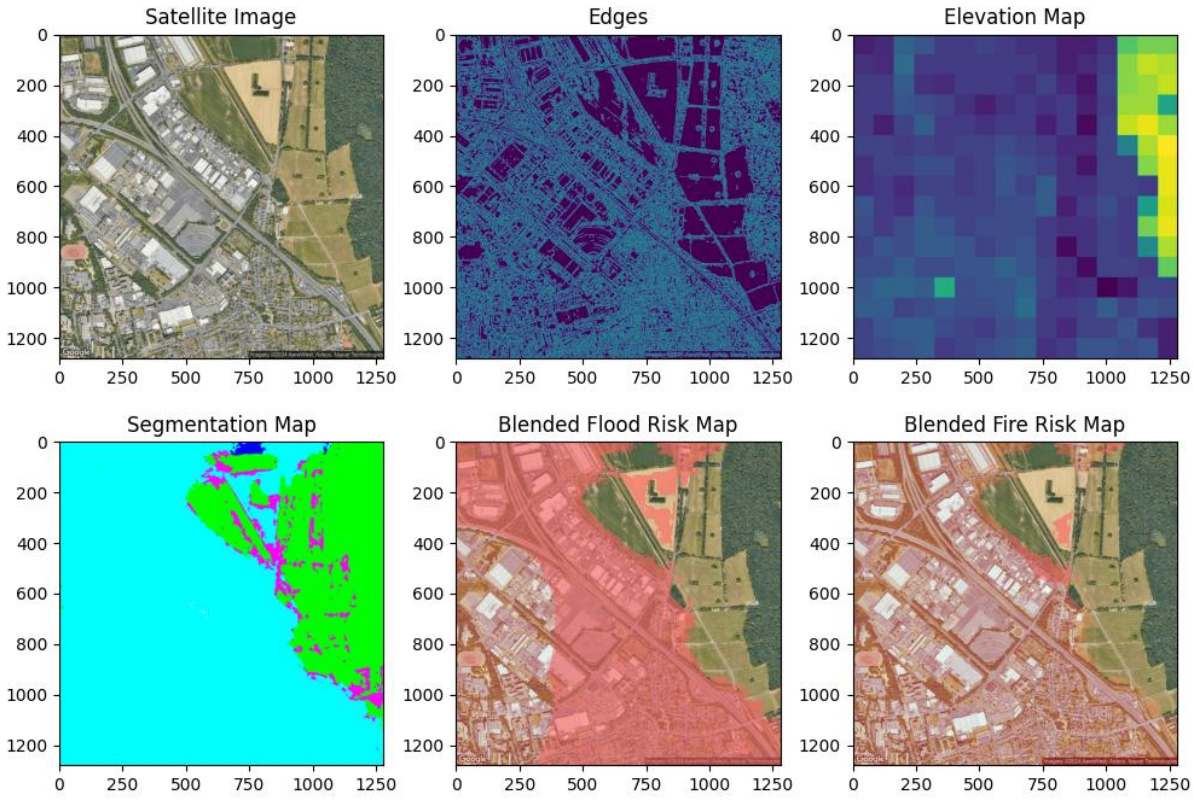


Figure 7: Cologne, Germany. Forests detected with some rangeland misclassified and two water false positives.

7.1.3 Image retrieval

The Convolutoional Autoencoder was trained for 20 epoches on the whole dataset using Adam optimizer and Mean Squared Loss function. The final MSELoss value was 0.0037.

The Nearest Neighbour algorithm was used to retrieve the first k=50 similar images to the input, using cosine similarity to compare feature maps. Then we computed distance between color histograms using KL divergence, obtaining a ranking based on similarity between the k histograms and the input's histograms. We took the first n=12 images from this ranked list. The histogram comparison has been added to improve the overall result of the retrieval, exploiting the fact that usually similar environments are characterized by similar color distributions. In our tests, we were able to successfully perform satellite imagery retrieval using input images with different types of environment, such as cultivated fields, urban areas or range land.

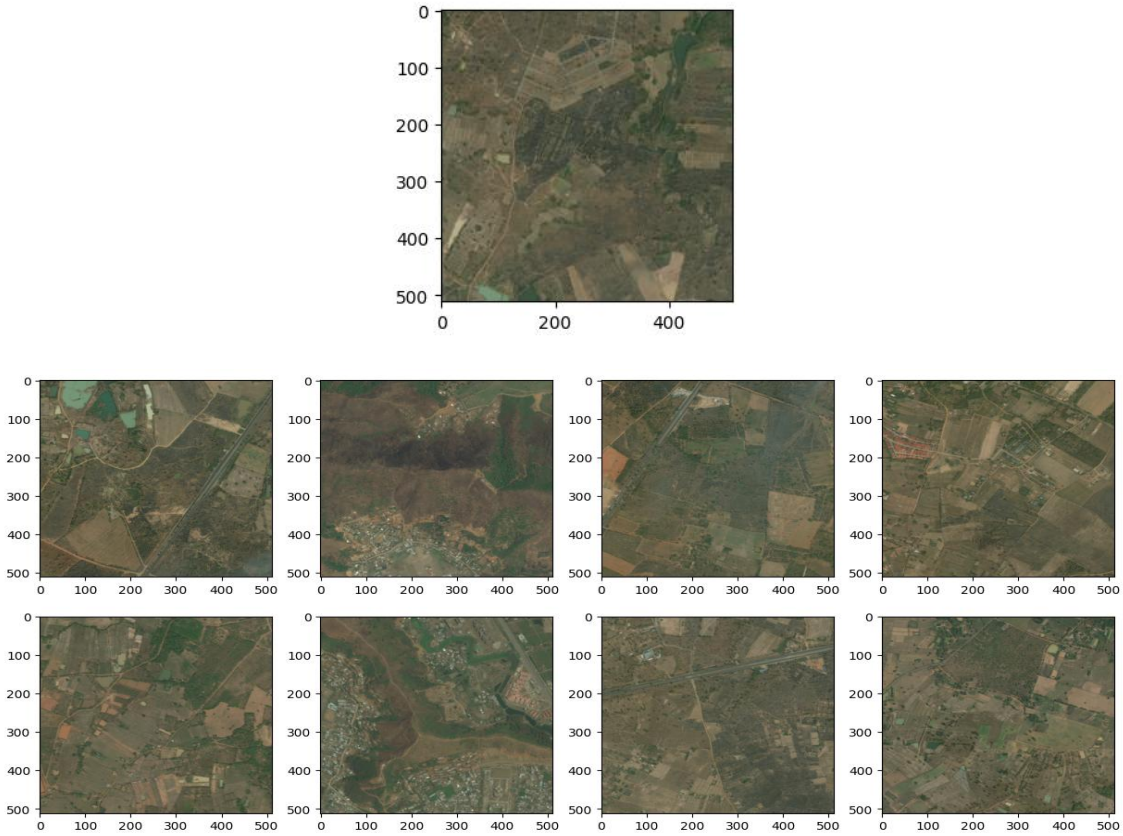


Figure 8: retrieval-output

8. Results and conclusions

In conclusion, in this paper we explored some possible solutions to automated satellite imagery analysis. We presented a possible approach to deal with satellite images in an unsupervised manner, and demonstrated how different techniques such as edge detection and segmentation could be integrated and how the Segformer model could be adapted to work with our specific type of input.
But based on the results obtained during the various trainings, we can conclude that adding the edge channel during the semantic segmentation phase did not bring a significant improvement compared to the original Segformer model.
There could still be some possible avenues of improvement as during the final testing phases we noticed that the model had difficulty dealing with different brightness levels present in google earth images, which could have been avoided by further augmenting the dataset with randomized brightness changes.

9. References

[1] Demir, I.; Koperski, K.; Lindenbaum, D.; Pang, G.; Huang, J. Basu, S.; Hughes, F.; Tuia, D.; Raskar, R. DeepGlobe 2018: A Challenge to Parse the Earth Through Satellite Images. The IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, 2018.

[2] Wang, J.; Zheng, Z.; Ma, A.; Lu, X.; Zhong, Y. LoveDA: A Remote Sensing Land-Cover Dataset for Domain Adaptive Semantic Segmentation. Zenodo, 2021.

[3] Xie, E. Z.; Wang, W. H.; Yu, Z. D.; Anandkumar, A.; Álvarez, J.; Luo, P. SegFormer: Simple and efficient design for semantic segmentation with transformers. arXiv preprint arXiv:2105.15203, 2021.

[4] Osman, S.A.; Das, J.; GIS-based flood risk assessment using multi-criteria decision analysis of Shebelle River Basin in southern Somalia. *SN Appl. Sci.* 5, 134 (2023). https://doi.org/10.1007/s42452-023-05360-5

[5] Gai, C.; Weng, W.;Yuan, H.; "GIS-Based Forest Fire Risk Assessment and Mapping," *2011 Fourth International Joint Conference on Computational Sciences and Optimization*, Kunming and Lijiang City, China, 2011, pp. 1240-1244, doi: 10.1109/CSO.2011.140.

[6] Canny, John. (1986). A Computational Approach To Edge Detection. Pattern Analysis and Machine Intelligence, IEEE Transactions on. PAMI-8. 679 - 698. 10.1109/TPAMI.1986.4767851.